

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5112

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I Bhavya PS(192524017) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Annexure A

Debugging & Optimisation Task

1. Debugging Hash Function Implementation (SHA Family)

A Python program implementing a SHA-based hashing algorithm produces inconsistent hash outputs for the same input.

Identify errors in message padding and block processing steps.
Fix issues related to bitwise operations and endianness handling.
Optimize the implementation to improve processing speed for large files (≥ 50 MB).
Evaluate how secure hash functions improve integrity compared to basic checksum methods.

2. Optimization of Secure Socket Layer (SSL/TLS) Simulation

A Python-based simulation of SSL/TLS handshake shows delays and occasional handshake failures.

Debug issues in certificate validation and key exchange steps.
Optimize the handshake process to reduce latency.
Refactor the code to reuse session keys efficiently (session resumption).
Analyze the performance vs security trade-offs in TLS optimization.

3. Debugging and Optimizing Digital Signature Algorithm (DSA)

A Python implementation of DSA generates invalid signatures during verification.

Identify issues in parameter selection and random number generation.
Fix errors in signature generation and verification steps.
Optimize modular arithmetic operations for faster execution.
Evaluate the impact of randomness quality on signature security.

4. Debugging Key Management System

A Python-based key management system fails to securely store and retrieve cryptographic keys.

Identify flaws in key storage (plaintext storage, improper encryption).
Fix issues related to key lifecycle management (generation, distribution, rotation).
Optimize secure storage using encryption and access control mechanisms.

Analyze how poor key management can compromise an entire cryptographic system.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

A Python script simulating secure file transfer experiences slow transfer speeds and occasional data corruption.

Debug issues in encryption/decryption synchronization.

Fix errors in packet handling and integrity verification.

Optimize throughput for large file transfers (≥ 100 MB).

Compare the optimized system with insecure file transfer methods and justify the improvements.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated

Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation
--------------	-----	---	--	---	---

1. Debugging Hash Function Implementation (SHA Family)

A. Problem Identification

A SHA-based hashing implementation should always produce the **same hash value** for the same input. If inconsistent outputs occur, the common causes are:

Errors Identified

1. Incorrect message padding
 - Padding bits (1 followed by zeros) are not added correctly.
 - Original message length is not appended as a 64-bit integer.
2. Incorrect block processing
 - Messages should be divided into fixed-size 512-bit blocks (SHA-256).
 - Missing or overlapping blocks cause incorrect hashes.
3. Bitwise operation errors
 - Wrong implementation of Rotate Right (ROTR).
 - Incorrect XOR, AND, NOT operations.
4. Endianness mismatch
 - SHA uses **big-endian** format.
 - Reading bytes in little-endian order changes the hash completely.

B. Debugging Steps

Step 1: Correct Message Padding

Incorrect:

```
message += b'\x80'
```

Correct:

```
message += b'\x80'
```

```
while (len(message) * 8) % 512 != 448:  
    message += b'\x00'  
message += original_length.to_bytes(8, 'big')
```

Step 2: Correct Block Processing

```
for i in range(0, len(message), 64):  
    block = message[i:i+64]
```

Each block must contain exactly **64 bytes (512 bits)**.

Step 3: Fix Rotate Right

Incorrect

```
x >> n
```

Correct

```
((x >> n) | (x << (32 - n))) & 0xffffffff
```

Step 4: Fix Endianness

Correct

```
int.from_bytes(block, 'big')
```

C. Optimization

Use Buffered File Reading

Instead of

```
data = file.read()
```

Use

```
while chunk := file.read(8192):  
    sha.update(chunk)
```

Benefits

- Lower RAM usage
- Faster processing
- Suitable for files larger than 50 MB

Avoid Repeated Calculations

Store frequently used values.

Example

```
sigma0 = ROTR(x,7)
```

instead of recalculating.

Use Built-in hashlib

```
import hashlib
```

```
sha = hashlib.sha256()
```

Python's implementation is optimized in C.

D. Analysis

SHA Hash	Checksum
Cryptographically secure	Not secure
Collision resistant	Easy collisions
Detects malicious modifications	Detects accidental errors only
One-way function	Easily forgeable

Conclusion

SHA ensures strong integrity protection while checksums only detect transmission errors.

2. Optimization of SSL/TLS Simulation

A. Problem Identification

Common Issues

- Expired certificates
- Invalid Certificate Authority
- Incorrect public key exchange
- Handshake timeout
- Incorrect cipher suite negotiation

B. Debugging

Certificate Validation

```
if certificate.expired:  
    raise Exception("Certificate Expired")
```

Verify

- Expiration date
- Certificate chain
- Hostname
- Digital signature

Key Exchange

Ensure both client and server compute identical shared secrets.

Incorrect

```
client_key != server_key
```

Correct

```
shared_secret = ECDH(private_key, public_key)
```

C. Optimization

Session Resumption

Instead of

Client



Full TLS Handshake



Server

Use

Client



Session Ticket



Server

Benefits

- Faster reconnection
- Lower CPU usage
- Reduced latency

Use ECDHE

Elliptic Curve Diffie-Hellman

Advantages

- Smaller keys
- Faster computations
- Forward secrecy

D. Performance vs Security

Optimization	Benefit	Risk
Session Resumption	Faster	Session ticket theft
ECDHE	Faster	Slightly higher implementation complexity
Certificate Caching	Reduced validation time	Requires cache management

Conclusion

TLS optimization should never weaken authentication or encryption.

3. Debugging Digital Signature Algorithm (DSA)

A. Problem Identification

Invalid signatures occur because of

- Weak prime parameters
- Incorrect generator
- Invalid private key
- Poor random number generation
- Wrong modular arithmetic

B. Debugging

Generate secure parameters

Large prime p

Prime q

Generator g

Random Number

Incorrect

```
random.randint()
```

Correct

```
import secrets
```

```
k = secrets.randbelow(q)
```

Signature

Correct

$$r = (g^k \bmod p) \bmod q$$

$$s = k^{-1}(H(m) + xr) \bmod q$$

Verification

$$w = s^{-1} \bmod q$$

$$u1 = H(m)w$$

$$u2 = rw$$

$$v = ((g^{u1} \times y^{u2}) \bmod p) \bmod q$$

Verify

$$v == r$$

C. Optimization

Use Python's efficient modular exponentiation

`pow(base, exponent, modulus)`

instead of

`(base ** exponent) % modulus`

Use Extended Euclidean Algorithm

instead of repeated division.

D. Analysis

Randomness quality directly affects DSA security.

Poor randomness



Repeated nonce



Private key recovery



Complete compromise

Modern implementations use secrets instead of random

4. Debugging Key Management System

A. Problem Identification

Problems

- Plaintext keys
- Hardcoded keys
- No rotation
- Unlimited access
- Missing backups

B. Debugging

Never store

secretkey123

Instead

Encrypt



Secure Database



Restricted Access

Generate keys

```
from secrets import token_bytes  
key = token_bytes(32)
```

Rotate Keys

Example

Every 90 Days

Old key



Archive



Generate New Key

C. Optimization

Store keys using

- AES-256 encryption
- Hardware Security Module (HSM)
- Access Control Lists
- Role-Based Access Control (RBAC)

Advantages

- Fast retrieval
- Secure storage
- Limited access

D. Analysis

Poor key management compromises

- Encryption
- Digital signatures
- Authentication
- Secure communication

Even AES-256 becomes useless if attackers steal the encryption key.

5. Optimization of Secure File Transfer Protocol (SFTP-like System)

A. Problem Identification

Common issues

- Packet loss
- Incorrect encryption order
- Synchronization failure
- Missing integrity checks
- Buffer overflow

B. Debugging

Encryption

Sender

Encrypt



Transmit



Receiver

Decrypt

Both sides must use

- Same key
- Same IV
- Same cipher mode

Integrity

Generate

SHA-256 Hash

Receiver

Recalculate hash



Compare



Accept or Reject

Packet Handling

Include

- Sequence Number
- Packet Length
- Checksum
- Authentication Tag

This prevents packet reordering.

C. Optimization

Transfer

100 MB+

Use

Buffered Streaming

```
chunk = file.read(65536)
```

instead of

```
file.read()
```

Use

AES-GCM

Benefits

- Encryption
- Authentication
- High speed

Use Multithreading

Thread 1

Read File

↓

Thread 2

Encrypt

↓

Thread 3

Transmit

This improves throughput.

D. Comparison

Secure SFTP	Normal FTP
Encrypted	Plaintext
Authentication	None
Integrity Verification	No
Secure Key Exchange	No
Resistant to Eavesdropping	Vulnerable

Justification

The optimized SFTP-like system provides:

- **Confidentiality** through strong encryption (e.g., AES-GCM).
- **Integrity** using authenticated encryption or SHA-256 verification.
- **Authentication** via secure key exchange and user verification.
- **High performance** by using chunked transfers, buffering, and parallel processing for files of 100 MB or more.